

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Чувашский государственный университет им. И.Н. Ульянова»

Кафедра вычислительной техники

Лабораторная работа № 4

По дисциплине “Операционные системы”

Выполнил: студент группы ИВТ-42-23

Иванов И.Л.

Проверил: кандидат технических наук

Ковалев С.В.

Чебоксары, 2026

Цель: Изучить выполнение операций команды активного процесса центрального процессора на основе Фон-Неймановской архитектуры ЭВМ.

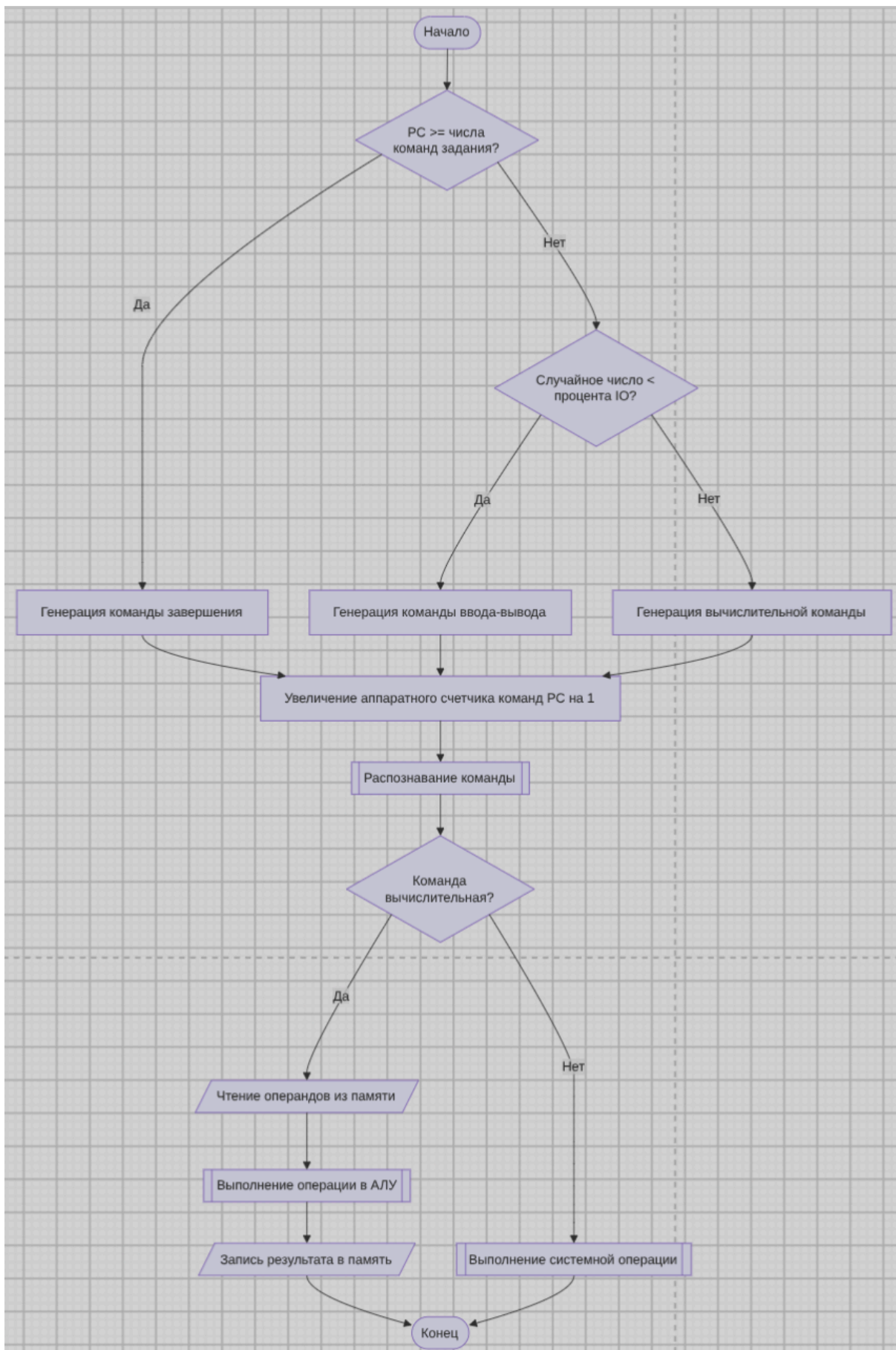
Исходники: https://github.com/kukirmash/os_model/

Задание

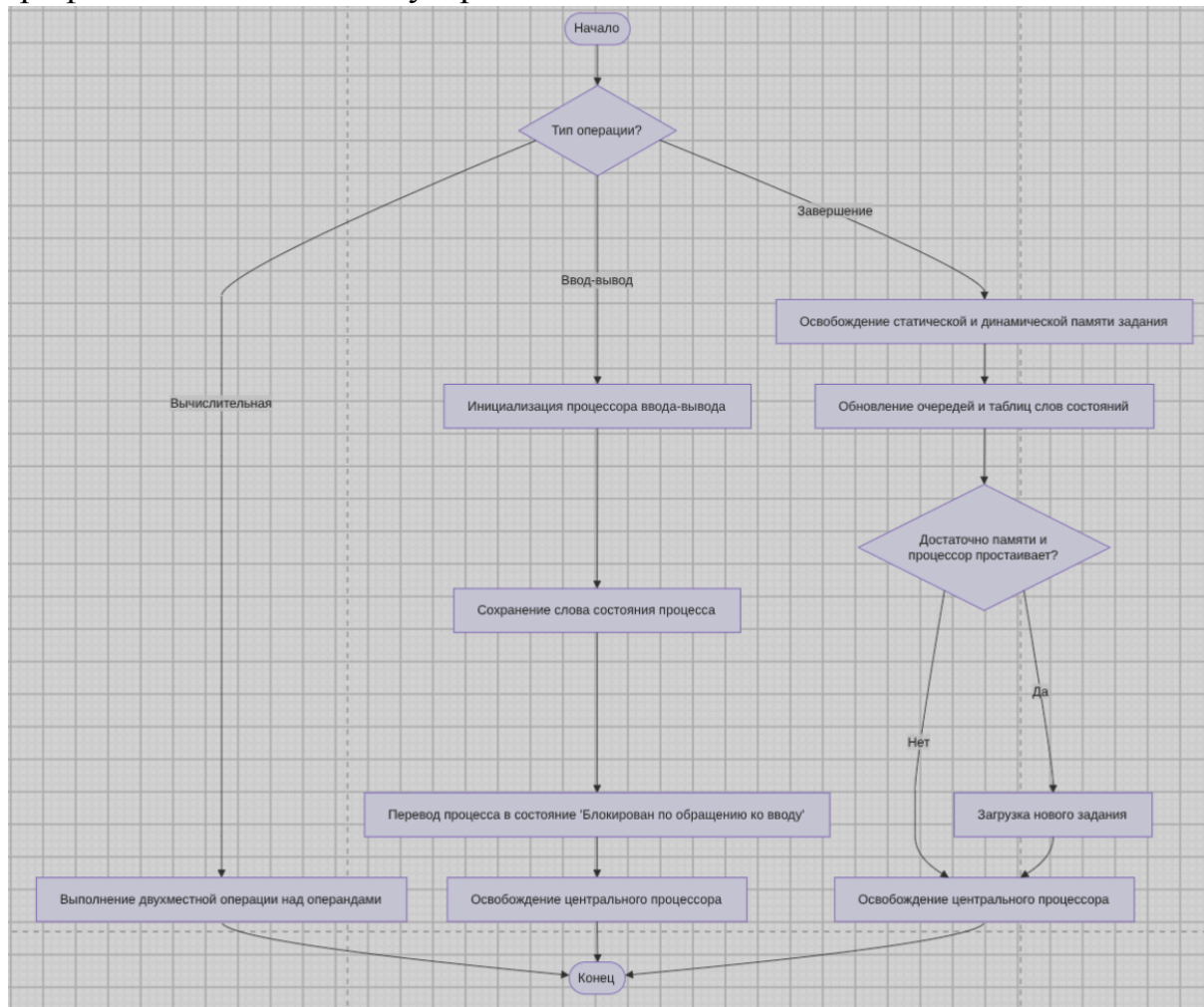
1. Изучить выполнение операций команды активного процесса центрального процессора на основе Фон-Неймановской архитектуры ЭВМ.
2. Добавить состояние «Блокирован по обращению ко вводу (выводу)». Список возможных состояний должен включать: «Готов», «Активен», «Инициализация IO», «Блокирован».
3. Реализовать подпрограмму «Выполнение команды процесса» с использованием следующих подпрограмм: «Генерация команды процесса», «Разборка команды и получение кода операции», «Чтение операнда из памяти», «Выполнение операции», «Инициализация процессора ввода (вывода)», «Завершение процесса» (с загрузкой одного или нескольких новых заданий), «Запись результата операции в память».
4. Изменить подпрограмму индикации, добавив вывод типа выполняемой команды.

БСА:

Алгоритм работы модуля моделирующего работу с памятью.



Алгоритм работы модуля моделирующего работу с арифметико-логического устройства.



Анализ кода [cpu.go](#)

Эта функция имитирует один машинный такт процессора. Сначала она читает (генерирует) команду с помощью `cpu.Fetch()`. Затем происходит инкремент аппаратного счетчика команд (`PC++`), так как длина команды в данной модели всегда равна 1. Функция вызывает `cpu.Decode()` для распознавания типа считанной команды. Завершается такт вызовом функции `cpu.Execute()`, которая читает операнды, выполняет операцию и записывает результат

```
func (cpu *CPU) Tick() {
    if cpu.State != StateWorking || cpu.ActiveProcess == nil {
        return
    }

    // 1. Чтение команды из памяти по адресу на аппаратном счётчике
    команд.
    cpu.CurrentCommand = cpu.Fetch()
```

```

    // 2. Увеличение аппаратного счётчика команд на длину команды (равна
1).
    cpu.ActiveProcess.PC++

    // 3. Распознавание команды.
    cpu.CurrentCommand.Type = cpu.Decode(cpu.CurrentCommand)

    // 4-6. Чтение операндов, выполнение операции, запись результата.
    cpu.Execute(cpu.CurrentCommand)
}

```

Функция отвечает за загрузку процесса в процессор. В модели всё состояние процесса хранится в его структуре слова состояния (PSW), поэтому отдельное сохранение старого контекста не требуется. Если передан пустой указатель (nil), процессор переходит в режим ожидания (StateIdle). Если передан корректный процесс, его состояние меняется на активное (process.StateActive), а состояние процессора переключается на рабочий режим (StateWorking).

```

func (cpu *CPU) ContextSwitch(p *process.PSW) {
    cpu.ActiveProcess = p
    if p == nil {
        cpu.State = StateIdle
        return
    }

    p.State = process.StateActive
    cpu.State = StateWorking
}

```

Данный метод имитирует чтение команды из памяти по аппаратному счетчику команд. В целях симуляции команды генерируются случайным образом "на лету". Если счетчик команд (PC) активного процесса достиг или превысил общее число команд задания (TotalCommands), выдается команда на завершение (OpTerminate). С заданной вероятностью (IOPercent) генерируется системная команда ввода-вывода (OpIO) со случайной продолжительностью выполнения. Если команда не системная, случайным образом выбирается одна из вычислительных операций (сложение, вычитание, умножение, деление) со случайными математическими адресами операндов в пределах памяти.

```

func (cpu *CPU) Fetch() Command {
    ap := cpu.ActiveProcess

    if ap == nil || ap.Task == nil {
        return Command{Code: OpTerminate}
    }
}

```

```

// Завершающая команда, процесс полностью выполнен
if ap.PC >= ap.Task.TotalCommands {
    return Command{Code: OpTerminate}
}

// Команда ввода-вывода
if rand.IntN(100) < ap.Task.IOPercent {
    return Command{Code: OpIO, Duration: cpu.newIODuration()}
}

// Вычислительная команда: код операции и математические адреса
операндов.
computeOps := [...]Operation{OpAdd, OpSub, OpMul, OpDiv}
return Command{
    Code:      computeOps[rand.IntN(len(computeOps))],
    Address1:  rand.IntN(OperandMemorySize),
    Address2:  rand.IntN(OperandMemorySize),
}
}

```

Метод реализует непосредственное исполнение команды в зависимости от её типа. Для вычислительных команд процессор читает два операнда из аппаратной памяти, выполняет над ними операцию с помощью функции **applyOperation**, а затем записывает полученный результат обратно в память по адресу второго операнда. Для команды ввода-вывода процессор обращается к интерфейсу подсистемы ввода-вывода, передавая ей процесс и длительность операции, после чего текущий активный процесс снимается с процессора, и CPU переходит в режим ожидания. Для команды завершения происходит освобождение памяти, занимаемой заданием, состояние процесса сбрасывается, а процессор переходит в статус StateIdle.

```

func (cpu *CPU) Execute(cmd Command) {
    ap := cpu.ActiveProcess
    if ap == nil {
        return
    }

    switch cmd.Type {
    case CommandCompute:
        operand1 := cpu.ReadOperand(cmd.Address1)

```

```

        operand2 := cpu.ReadOperand(cmd.Address2)
        cpu.WriteOperand(cmd.Address2, applyOperation(cmd.Code,
operand1, operand2))

    case CommandIO:
        cpu.IO.StartIO(ap, cmd.Duration)
        cpu.ActiveProcess = nil
        cpu.State = StateIdle

    case CommandTerminate:
        cpu.Memory.Free(*ap.Task)
        ap.State = process.StateNone
        cpu.ActiveProcess = nil
        cpu.State = StateIdle
}
}

```

Листинг программы:

```

// Package cpu моделирует центральный процессор и АЛУ.
//
// Архитектура фон-неймановская: команды и данные хранятся в общей
памяти,
// команда читается по аппаратному счётчику команд (PC). Процессор
работает
// только с одним текущим активным процессом.
package cpu

import (
    "math/rand/v2"

    "os_model/internal/process"
)

//
-----
// State — состояние процессора.
type State int

const (
    StateIdle    State = iota // ожидание: активного процесса нет
    StateWorking              // работа: выполняется команда активного
процесса
)

//
-----
// CommandType — тип команды после декодирования.

```

```

type CommandType int

const (
    CommandCompute    CommandType = iota // вычислительная команда (АЛУ)
    CommandIO          // команда ввода-вывода
    CommandTerminate   // команда завершения процесса
)

//
-----

// Operation — код операции команды.
type Operation int

const (
    // вычислительные операции (выполняются в АЛУ)
    OpAdd Operation = iota
    OpSub
    OpMul
    OpDiv

    // системные
    OpIO      // обращение к устройству ввода-вывода
    OpTerminate // завершение задания
)

//
-----

// Command — команда, обрабатываемая процессором.
type Command struct {
    Type      CommandType // тип команды, распознанный при декодировании
    Code      Operation    // код операции
    Address1  int           // математический адрес первого операнда
    Address2  int           // математический адрес второго операнда
    (приёмник результата)
    Duration  int           // длительность операции ввода-вывода, тактов
}

//
-----

// IOManager — часть подсистемы ввода-вывода, используемая процессором
// для инициализации операции ввода-вывода.
type IOManager interface {
    StartIO(p *process.PSW, duration int)
}

//
-----

// MemoryManager — часть менеджера памяти, используемая процессором
// при завершении процесса.
type MemoryManager interface {
    Free(task process.Task)
}

```

```

//
-----

// OperandMemorySize – размер моделируемой аппаратной памяти под
операнды.
const OperandMemorySize = 256

// CPU – центральный процессор модели.
type CPU struct {
    ActiveProcess *process.PSW // указатель на PSW текущего активного
процесса
    State          State        // состояние процессора
    CurrentCommand Command      // команда, обрабатываемая в текущем
такте

    // Смежные подсистемы; проставляются ядром при инициализации.
    IO      IOManager // подсистема ввода-вывода
    Memory  MemoryManager // менеджер памяти

    // IOMaxDuration – максимальная длительность операции ввода-вывода,
тактов.
    IOMaxDuration int

    // OperandMemory – аппаратная память, в которой хранятся операнды
// (инициализируется лениво).
    OperandMemory []int
}

//
-----

// ContextSwitch сохраняет состояние текущего процесса (если он есть)
// и загружает PSW нового процесса, восстанавливая его состояние.
//
// В модели всё состояние процесса хранится в его PSW (включая PC),
// поэтому отдельное сохранение не требуется: при следующей загрузке
// поля восстанавливаются из PSW.
func (cpu *CPU) ContextSwitch(p *process.PSW) {
    cpu.ActiveProcess = p
    if p == nil {
        cpu.State = StateIdle
        return
    }

    p.State = process.StateActive
    cpu.State = StateWorking
}

//
-----

// Tick выполняет один такт процессора: выборку, декодирование и
выполнение
// одной команды активного процесса. Длина команды в модели равна 1,
// поэтому PC увеличивается сразу после выборки.
func (cpu *CPU) Tick() {
    if cpu.State != StateWorking || cpu.ActiveProcess == nil {
        return
    }

```

```

    }

    // 1. Чтение команды из памяти по адресу на аппаратном счётчике
команд.
    cpu.CurrentCommand = cpu.Fetch()

    // 2. Увеличение аппаратного счётчика команд на длину команды (равна
1).
    cpu.ActiveProcess.PC++

    // 3. Распознавание команды.
    cpu.CurrentCommand.Type = cpu.Decode(cpu.CurrentCommand)

    // 4-6. Чтение операндов, выполнение операции, запись результата.
    cpu.Execute(cpu.CurrentCommand)
}

//
-----
// Fetch читает команду из памяти по аппаратному счётчику команд.
//
// В модели команды не хранятся, а генерируются случайным образом:
// пока PC меньше общего числа команд задания, с вероятностью IOPercent
// генерируется команда ввода-вывода, иначе – вычислительная; когда PC
// достигает TotalCommands, генерируется команда завершения.
func (cpu *CPU) Fetch() Command {
    ap := cpu.ActiveProcess

    if ap == nil || ap.Task == nil {
        return Command{Code: OpTerminate}
    }

    // Завершающая команда, процесс полностью выполнен
    if ap.PC >= ap.Task.TotalCommands {
        return Command{Code: OpTerminate}
    }

    // Команда ввода-вывода
    if rand.IntN(100) < ap.Task.IOPercent {
        return Command{Code: OpIO, Duration: cpu.newIODuration()}
    }

    // Вычислительная команда: код операции и математические адреса
операндов.
    computeOps := [...]Operation{OpAdd, OpSub, OpMul, OpDiv}
    return Command{
        Code:      computeOps[rand.IntN(len(computeOps))],
        Address1:   rand.IntN(OperandMemorySize),
        Address2:   rand.IntN(OperandMemorySize),
    }
}

//
-----
// Decode распознаёт команду: вычислительная, ввод-вывод или
завершение.

```

```

func (cpu *CPU) Decode(cmd Command) CommandType {
    switch cmd.Code {
    case OpIO:
        return CommandIO
    case OpTerminate:
        return CommandTerminate
    default:
        return CommandCompute
    }
}

//
-----

// Execute выполняет операцию в АЛУ.
//
// Вычислительная команда выполняется за один такт: операнды читаются
// из аппаратной памяти, над ними выполняется двухместная операция,
// результат записывается обратно (во второй операнд). Команда
// ввода-вывода
// переводит процесс в состояние инициализации IO, после чего процессор
// освобождается. Команда завершения иницирует выгрузку процесса.
func (cpu *CPU) Execute(cmd Command) {
    ap := cpu.ActiveProcess
    if ap == nil {
        return
    }

    switch cmd.Type {
    case CommandCompute:
        operand1 := cpu.ReadOperand(cmd.Address1)
        operand2 := cpu.ReadOperand(cmd.Address2)
        cpu.WriteOperand(cmd.Address2, applyOperation(cmd.Code,
operand1, operand2))

    case CommandIO:
        cpu.IO.StartIO(ap, cmd.Duration)
        cpu.ActiveProcess = nil
        cpu.State = StateIdle

    case CommandTerminate:
        cpu.Memory.Free(*ap.Task)
        ap.State = process.StateNone
        cpu.ActiveProcess = nil
        cpu.State = StateIdle
    }
}

//
-----

// ReadOperand читает операнд из аппаратной памяти по математическому
адресу.
func (cpu *CPU) ReadOperand(addr int) int {
    cpu.ensureOperandMemory()
    if addr < 0 || addr >= len(cpu.OperandMemory) {
        return 0
    }
}

```

```

    return cpu.OperandMemory[addr]
}

//
-----

// WriteOperand записывает результат операции в аппаратную память.
func (cpu *CPU) WriteOperand(addr int, value int) {
    cpu.ensureOperandMemory()
    if addr < 0 || addr >= len(cpu.OperandMemory) {
        return
    }
    cpu.OperandMemory[addr] = value
}

//
-----

// ensureOperandMemory лениво (нулями) инициализирует аппаратную память
под операнды.
func (cpu *CPU) ensureOperandMemory() {
    if cpu.OperandMemory == nil {
        cpu.OperandMemory = make([]int, OperandMemorySize)
    }
}

//
-----

// newIODuration генерирует длительность операции ввода-вывода
// в диапазоне 1..IOMaxDuration тактов.
func (cpu *CPU) newIODuration() int {
    maxDuration := cpu.IOMaxDuration
    if maxDuration < 1 {
        maxDuration = 1
    }
    return 1 + rand.IntN(maxDuration)
}

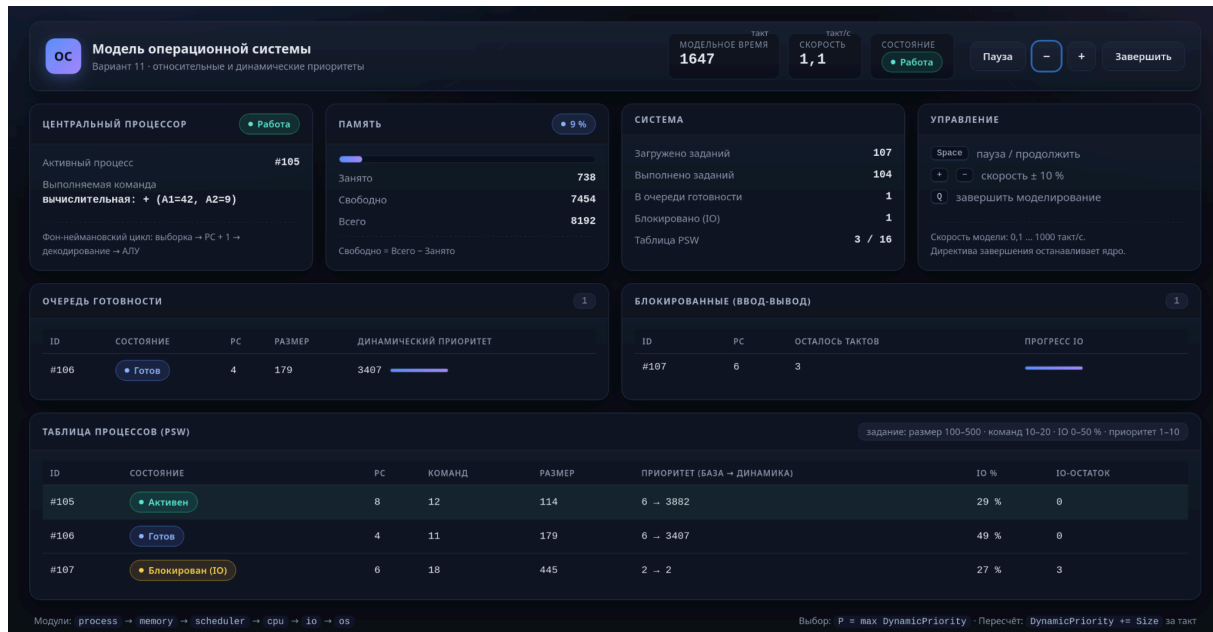
//
-----

// applyOperation моделирует работу АЛУ, выполняя двухместную операцию.
func applyOperation(code Operation, operand1, operand2 int) int {
    switch code {
    case OpAdd:
        return operand1 + operand2
    case OpSub:
        return operand1 - operand2
    case OpMul:
        return operand1 * operand2
    case OpDiv:
        if operand2 == 0 {
            return 0
        }
        return operand1 / operand2
    default:
        return operand2
    }
}

```



Результат:



Вывод: в ходе данной работы были изучены выполнение операций команды активного процесса центрального процессора на основе Фон-Неймановской архитектуры ЭВМ и получены практические навыки.